SAVIRACORTEX

Technical Report • Palm Mind AI Assessment

01 • OVERVIEW

Executive Summary

SaviraCortex is a knowledge-oriented backend built around a Retrieval-Augmented Generation (RAG) workflow. The system's central goal is to make supplied documents or knowledge resources usable for question answering: first transforming source material into searchable representations, then retrieving relevant evidence when a user asks a question.

What the project addresses

Language models can answer fluently while lacking access to private, newly supplied, or domain-specific information. A retrieval layer helps ground responses in source content rather than relying only on model parameters. SaviraCortex separates knowledge preparation from question-time retrieval, allowing content to be indexed and then queried through a backend service.

Core lifecycle

The high-level lifecycle is: source submission → ingestion and text preparation → indexing/storage → user query → contextual retrieval → response construction. The ingestion pipeline prepares material ahead of requests; the retrieval pipeline runs when a question arrives.

The project is best understood as a backend RAG service: ingestion makes knowledge searchable, retrieval selects relevant context, and the API exposes the workflow to a client.

Assessment context

The project was developed for the Palm Mind AI assessment. A hosted service URL has been shared: https://palmmindai-assesment.onrender.com/. The link is included as the deployment reference; this report does not claim a live health check or independently verified endpoint behavior.

Report scope

This document describes the conceptual and discussed architecture, practical workflow, API/deployment framing, quality considerations, and extensions. Where exact library names, endpoint schemas, chunk parameters, embedding models, or storage configuration were not confirmed in the available project discussion, they are intentionally left unspecified.

02 • MOTIVATION

Problem Statement & Objectives

Problem statement

Knowledge is often stored in files or documents that are not directly accessible to a generative model at answer time. A useful application must extract content, organize it into searchable units, and retrieve the most relevant information for a question. Without this retrieval process, responses may be generic, incomplete, or unsupported by the source material.

Project objectives

• Build a backend flow that accepts knowledge material and prepares it for downstream retrieval.

• Make prepared content searchable in response to natural-language questions.

• Provide an API surface so a client or external application can interact with the system.

• Keep ingestion and retrieval as identifiable stages, simplifying reasoning about failures and improvements.

• Support a deployment-ready project experience, including a hosted endpoint reference.

Why RAG

RAG combines retrieval with generation. Retrieval supplies a context window containing relevant source passages; generation uses that context to formulate a response. This architecture can improve grounding, make knowledge updates more practical than retraining a model, and support source-specific question answering.

Expected user-facing value

A user can ask questions against a prepared knowledge collection instead of manually searching every document. The service can potentially be reused for internal knowledge assistants, document exploration, assessment demonstrations, and domain-specific Q&A.;

A key design principle is to keep the retrieved evidence connected to the answer-generation step, so response quality can be evaluated against source material.

03 • ARCHITECTURE

System Architecture at a Glance

Logical architecture

At a conceptual level, SaviraCortex has two major processing paths: an offline or preparation-oriented ingestion path and an online query-oriented retrieval path. Both are connected by the indexed knowledge store.

Stage	Role	Output
Source input	Receive documents or knowledge assets	Raw source material
Ingestion	Extract and normalize content; prepare retrieval units	Clean text / chunks
Representation	Convert prepared units into searchable representations	Indexed records
Knowledge store	Persist records and retrieval metadata	Queryable corpus
Query handling	Accept and contextualize the user’s question	Search-ready query
Retrieval	Find relevant items from indexed knowledge	Selected context
Response layer	Use retrieved context to form a response	Answer payload

Data flow boundaries

Ingestion transforms source material into a form that retrieval can operate on. Retrieval uses the prepared corpus and does not need to repeat the entire ingestion process for every question. The API boundary allows clients to submit requests and receive results without directly managing internal processing.

Architecture note

The architecture is described at a logical level. Exact module names, runtime dependencies, database vendor, embedding model, and prompt structure should be confirmed from the repository before being presented as definitive implementation facts.

Separation of concerns: document preparation is distinct from query-time search and answer construction.

04 • KNOWLEDGE PREPARATION

Ingestion Pipeline

Purpose

The ingestion pipeline converts source knowledge into records that can be searched efficiently. It is a preparation stage: its work makes the knowledge collection available to future user queries.

Typical stages in a RAG ingestion workflow

Step	Description
1. Source intake	Receive and validate the source document or resource.
2. Text extraction	Extract textual content from the source format.
3. Normalization	Clean formatting artifacts and standardize the extracted text.
4. Segmentation	Divide long text into manageable retrieval chunks.
5. Metadata association	Attach available attributes such as document identity or location within the source.
6. Vectorization / indexing	Represent chunks in a search-compatible form and store them in the retrieval index.
7. Completion	Make prepared records available to the retrieval path.

Why chunking matters

A complete document may be too large or too broad to pass into a model for each query. Segmentation creates focused pieces that can be retrieved independently. Chunk boundaries, chunk size, and overlap affect whether the retrieved context preserves enough meaning.

Practical considerations

• Validate source inputs and handle extraction failures explicitly.

• Preserve metadata so retrieved passages can be traced to a source.

• Prevent duplicate ingestion from unnecessarily multiplying indexed content.

• Make ingestion status observable, especially for large or problematic sources.

• Consider re-ingestion/versioning when the original source changes.

The exact extraction libraries, chunk parameters, embedding configuration, and index implementation are not specified here because they were not confirmed in the discussion.

05 • QUESTION-TIME FLOW

Retrieval Pipeline & Query Processing

Purpose

The retrieval pipeline runs when a user submits a question. Its task is to locate context from the indexed corpus that is relevant to the query, then make that context available to the response-generation stage.

Discussed architectural evolution

The project discussion contrasted a legacy design that expanded a question into multiple contextual queries and dispatched workers, followed by global fusion, global ranking, and an additional aggregation step, with a simplified current direction. The current direction was described as limiting processing to a contextualized query rather than retaining the entire multi-query orchestration chain.

Simplified conceptual flow

Step	Responsibility
1. Query intake	Receive the user’s natural-language question.
2. Contextualization	Prepare a contextualized form of the question for retrieval.
3. Candidate retrieval	Search the indexed knowledge for relevant chunks.
4. Context selection	Choose suitable retrieved material for the answer stage.
5. Answer construction	Use selected context to formulate the response.

Why simplification can help

• Less orchestration overhead and fewer moving components.

• A more straightforward control flow that is easier to debug.

• Reduced complexity compared with parallel query workers and global fusion.

• Clearer opportunities to measure retrieval quality at individual stages.

This description reflects the design change discussed in conversation; exact retrieval algorithms and final implementation details should be validated against source code.

06 • SERVICE ACCESS

API & Hosted Deployment

Backend API role

The API provides an interface between a consuming client and the backend’s knowledge workflows. It allows callers to submit supported operations without needing to understand how documents are internally prepared or how relevant context is selected.

Deployment reference

Hosted URL: https://palmmindai-assesment.onrender.com/

API documentation and verification

For a technical handoff, the API’s exact routes, HTTP methods, request schemas, response formats, authentication requirements, and error codes should be documented from the live API specification or repository. This report does not invent endpoint names or example payloads.

Deployment considerations

• Configure secrets and provider credentials through environment variables rather than source control.

• Ensure startup initializes required services and dependencies safely.

• Expose useful health/readiness signals where supported.

• Log request failures without leaking secrets or sensitive document text.

• Consider service cold starts and resource limits on hosted infrastructure.

• Keep deployment configuration and local development configuration aligned.

A reproducible deployment handoff should include

Repository setup instructions, dependency installation, environment variable reference, startup command, endpoint documentation, and a small smoke-test procedure. The provided Render URL can serve as the public deployment reference for the assessment.

07 • QUALITY

Response Quality, Evaluation & Reliability

What good retrieval-grounded answers require

Answer quality depends on more than the language model. The system must ingest content correctly, retrieve passages that are relevant to the user's question, and use those passages appropriately during response construction.

Evaluation dimensions

Dimension	What it measures	Example check
Retrieval relevance	Are returned passages related to the question?	Review top retrieved chunks for representative queries.
Coverage	Does the context contain the needed facts?	Check whether required facts appear among candidates.
Faithfulness	Is the answer supported by retrieved evidence?	Compare answer claims with source passages.
Answer usefulness	Does the answer directly address the query?	Human review or a suitable evaluation rubric.
Robustness	Does behavior remain reasonable on edge cases?	Test vague, out-of-domain, and no-match questions.
Operational reliability	Can service failures be detected and handled?	Exercise invalid inputs, timeouts, and dependency errors.

Suggested test set

• Direct factual questions whose answers are present in a source.

• Questions requiring information spread across multiple passages.

• Queries using paraphrases or alternate terminology.

• Questions not supported by the knowledge collection.

• Empty, malformed, or overly broad input.

• Changed or re-ingested documents to check corpus freshness.

Reliability behavior

A robust system should avoid presenting unsupported claims as source-grounded facts. If relevant evidence is absent, the response layer should be able to communicate uncertainty or lack of support. The system should also return clear errors for invalid requests and recover gracefully from transient service dependencies.

08 • ENGINEERING PRACTICES

Security, Maintainability & Operations

Security and data handling

• Treat uploaded or ingested documents as potentially sensitive and define access controls appropriate to the deployment.

• Do not expose credentials, provider keys, or internal configuration in API responses or logs.

• Validate file types, sizes, and request payloads at the service boundary.

• Limit and sanitize metadata fields that may be displayed to clients.

• Document data retention and deletion behavior if the service stores user-provided content.

Maintainability

A maintainable RAG backend benefits from clear modules for input validation, ingestion, retrieval, response construction, and API routing. Keeping interfaces explicit makes it easier to change one stage without unintentionally changing the whole application.

Observability

Useful operational signals include request latency, ingestion duration, retrieval counts, failure rates, and dependency errors. Logs should support debugging while minimizing the recording of sensitive user prompts or source content.

Performance considerations

• Indexing work should be separated from repeated question-time work where practical.

• Retrieval candidate counts and context length should be tuned against quality and latency.

• Cache only when cache invalidation and data freshness are understood.

• Measure actual bottlenecks before adding parallelism or extra ranking layers.

Operational recommendations are proposed engineering practices, not claims that each item is already implemented in SaviraCortex.

09 • ROADMAP

Limitations, Trade-offs & Future Work

Current trade-offs

The discussed simplification from a multi-query worker-and-fusion flow to a contextualized-query approach reduces orchestration complexity. The trade-off is that multi-query expansion can sometimes discover complementary passages that a single query may miss. This should be assessed empirically using a representative test set rather than assumed.

Potential improvements

Area	Potential extension	Reason
Evaluation	Build a repeatable benchmark of query-answer pairs	Track retrieval and answer quality over revisions.
Retrieval	Tune query contextualization and candidate selection	Improve relevant evidence recall.
Ingestion	Add stronger duplicate/version handling	Keep the corpus consistent as sources change.
Traceability	Expose source references in answer payloads where suitable	Help users verify generated claims.
Observability	Metrics and structured logging	Make production behavior easier to diagnose.
API experience	Publish precise OpenAPI docs and examples	Improve integration for clients and reviewers.
Reliability	Define no-evidence and dependency-failure behavior	Avoid misleading or opaque responses.

Prioritization

A practical next step is to first establish a small baseline evaluation set and capture retrieval outputs. Then tune the current simplified flow. Additional orchestration components—such as multi-query generation, fusion, or reranking—should be reintroduced only if measured results show meaningful improvement relative to added latency and complexity.

Recommended principle: optimize the simplest architecture that meets the measured quality target.

10 • SUMMARY

Conclusion & Project Handoff

Conclusion

SaviraCortex presents a backend RAG workflow intended to make supplied knowledge available for natural-language question answering. The project separates ingestion—preparing and indexing source material—from retrieval—the query-time process that selects evidence. It also has a hosted deployment reference for assessment demonstration.

Key takeaways

• The project centers on a document/knowledge ingestion and retrieval backend.

• Ingestion prepares source material into searchable units.

• Retrieval contextualizes a question and searches prepared knowledge for relevant evidence.

• The discussed current design favors a contextualized-query approach over the legacy multi-query worker, fusion, ranking, and aggregation chain.

• The service is deployed at the shared Render URL, which is included as a reference rather than a verified uptime claim.

• Quality, grounding, reliability, and clear API documentation are natural focus areas for continued development.

Handoff checklist

• Confirm exact API endpoints and payloads against the source or OpenAPI docs.

• Record the technology stack, models, vector-store details, and configuration values from the repository.

• Include setup and run commands in the project README.

• Run a smoke test against the deployed service and capture results.

• Maintain a small evaluation dataset to compare retrieval behavior over time.

SaviraCortex — developed for the Palm Mind AI assessment.

Deployment reference: https://palmmindai-assesment.onrender.com/

End of report.